Student Marketplace

Assignment 1 Design and WDLC Note

Annan Khan | CMS 506644 | BESE15-B

Assigned idea 16 | Client-side web application | Due 30 October 2026

User and problem

Students need a single place to find goods, offer small services and request items from other students. Student Marketplace groups these activities into Goods, Services and Wanted, with category and price controls that reduce the effort of finding a relevant listing. All records are fictional and no payments are processed.

Core workflow

Browse or search listings, refine the results, open a listing and save it for later. A demo student can create a listing, review its live preview and submit it. New and edited posts enter Pending review in My listings. The demo owner can edit or delete those records; public browse shows active records only. A page refresh restores the supplied fixtures.

Assignment 1 scope

The application runs in the browser using semantic HTML, CSS and vanilla JavaScript. Meaningful interactions include combined search and filtering, in-memory listing CRUD, saved items and type-specific forms. Nine fixture records include six active listings and three owner records illustrating pending, rejected and expired states. Those states are demonstrations, not a working moderator service.

Approved visual design

The design uses a stark white page and card background, a blackletter masthead, compact slab-serif headlines and plain interface text. Narrow editorial columns contain occasional two-column features. Content-height packing allows subsequent listings to occupy space below a shorter feature. Wanted requests use a separate classifieds rail. On smaller screens the rail moves below the main listings and the layout reduces to a single column.

Goods use green solid rules and a filled text label. Services use blue double rules and an outlined label. Wanted uses rust dashed notices. Text and border patterns communicate type without relying on color alone. The form and other views use the same typography and spacing.

Wireframes and responsive structure

These schematics describe the approved layout and content hierarchy. They are wireframes, not browser screenshots. The running application remains the source for exact typography and spacing.

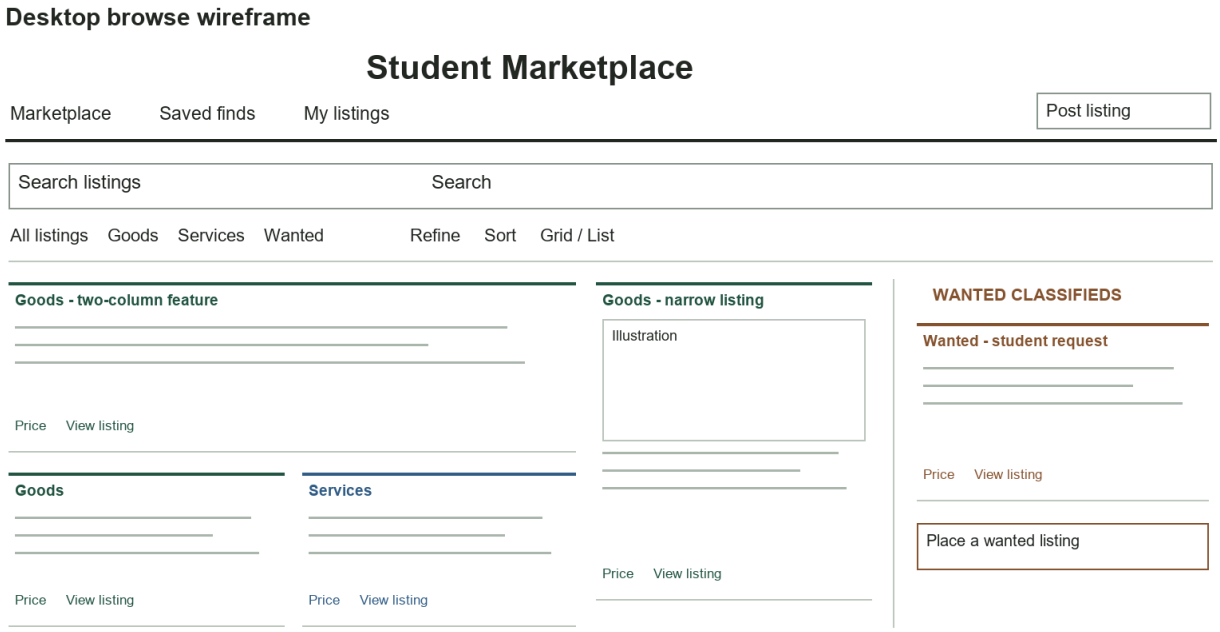


Figure 1 Desktop browse with occasional wider entries and a separate Wanted classifieds column

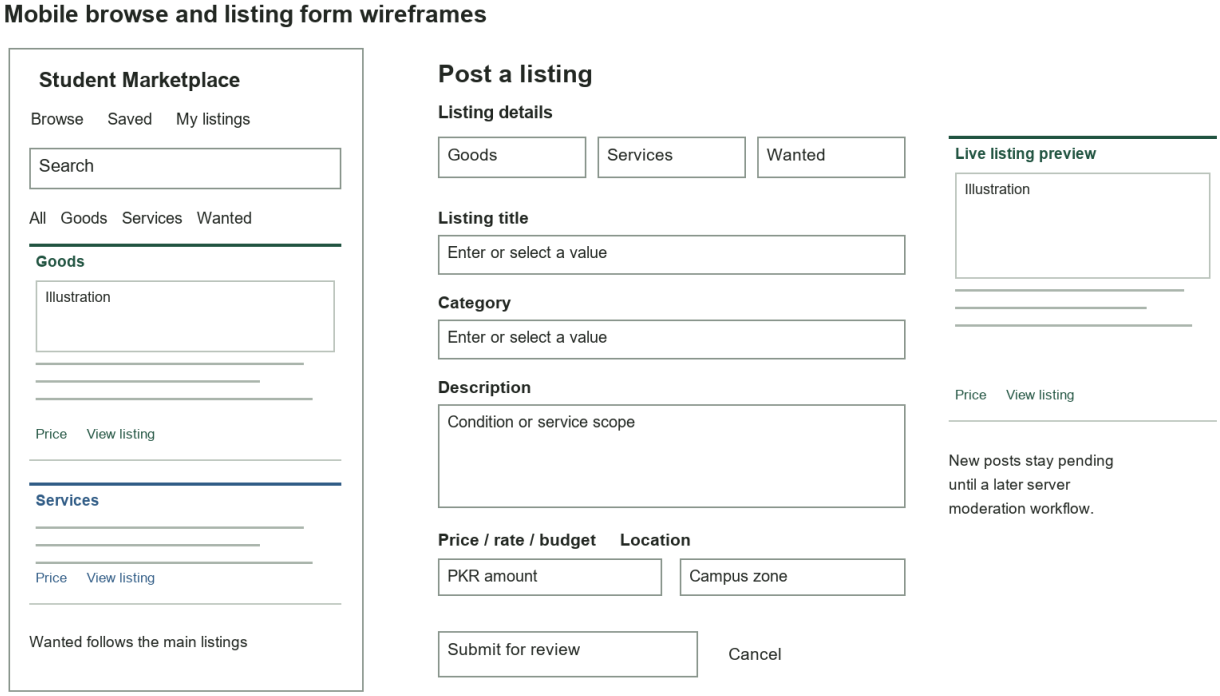


Figure 2 Mobile browse stacks vertically and the posting form provides labels, validation and a live preview

The CSS contains breakpoints for desktop, tablet and mobile layouts. Native headings, a skip link, input labels, fieldsets, visible focus states and ordinary links/buttons provide the intended keyboard and screen-reader structure. Desktop, tablet and mobile browser checks found no horizontal overflow. Heading-to-search keyboard focus was also verified.

Client design and validation

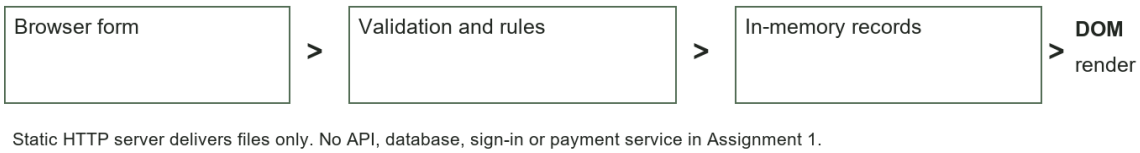


Figure 3 Assignment 1 data flow is local to the browser after static files load

Browser and server responsibilities

The static HTTP server returns HTML, CSS, JavaScript, images and fonts. The browser holds the listing array, filter state, saved-item set and recently viewed identifiers in memory. Event handlers validate inputs and update the DOM. No API call stores a record, and browser storage is not used for personal data. Later stages add server validation, persistence, authenticated ownership and moderation; the current client checks are not security controls.

Core sample records

Listing	Type	Amount
14-inch laptop	Goods	PKR 62,000
Engineering textbooks	Goods	PKR 2,800
Adjustable desk lamp	Goods	PKR 1,800
Calculus tutoring	Services	PKR 750 per hour
Unopened headphones	Goods	PKR 4,500
Study chair request	Wanted	Flexible budget

Validation and output safety

Browser constraints require a title of 5 to 80 characters, a description of 20 to 1500 characters, a valid category and an amount between 0 and 1,000,000 PKR in steps of 0.01. JavaScript repeats the checks for the application flow. Goods select a condition, services select a rate unit, and a flexible wanted budget disables the amount field. A minimum filter amount above the maximum produces an error. Empty results offer a clear reset action.

User-controlled text is escaped before template rendering or assigned through textContent. New and edited listings remain pending. Edit and delete actions check the fictional owner flag; a delete confirmation allows cancellation. Real privacy and access enforcement require the later server-backed implementation.

WDLC and verification

Web development lifecycle

Requirements: use idea 16, a browser-only application, responsive CSS, semantic forms, six or more records and meaningful interactions. Design: define the three listing types, core journey, wireframes, white newspaper layout and responsive behavior. Implementation: build the HTML shell, CSS layers, fixture data and event-driven JavaScript. Testing: exercise DOM workflows and edge cases, then inspect real viewports and keyboard use. Deployment and maintenance: publish versioned source, retain exact local run instructions and carry the same user problem into the PHP/PDO and MERN stages.

Automated verification

Check group	Result
Native constraints and amount limits	Passed
Search, empty results and invalid ranges	Passed
Create, edit, delete and pending visibility	Passed
Invalid submission and safe literal text	Passed
Save and unsave without persistent storage	Passed
Layout spacing calculation	Passed
Delete cancellation and non-owned target	Passed
Service units and flexible wanted budget	Passed

Eight Node and jsdom tests pass. Browser checks confirm native validation, create and edit, saved items, empty search, heading-to-search focus and three responsive sizes without horizontal overflow. Screenshots are in docs/screenshots. The Delete confirmation stalled in browser automation; deletion passes the DOM tests. A student live walkthrough is still required. The new fixes began with failing tests; the original prototype was not developed test first.

Repository and local demonstration

Repository: https://github.com/crankysmh47/student-marketplace-cs344

From the repository root, run the command below with Python 3 installed. Open http://127.0.0.1:5519/?theme=forest#browse and keep the terminal open. No backend or database setup is required.

```
py -m http.server 5519 --bind 127.0.0.1 --directory assignment-1
```

Continuity and attribution

Assignment 2 introduces PHP 8.5, PDO, persistence, Fetch, server validation and session/CSRF protection. Assignment 3 introduces React, Express, MongoDB and authenticated roles. University-email sign-in awaits the supplied template. Contact requests, real moderation, expiry processing and abuse reports are later-stage work.

The brief is CS344 Web Engineering Assignments 1 to 3 2026, supplied in the project folder. Original fictional SVG illustrations are bundled locally. Unifraktur Cook and Roboto Slab come from the official google/fonts repository with their SIL and Apache licenses. Earlier font variants retain their own notices in assets/fonts. The repository is private; the assessor must have access before review.